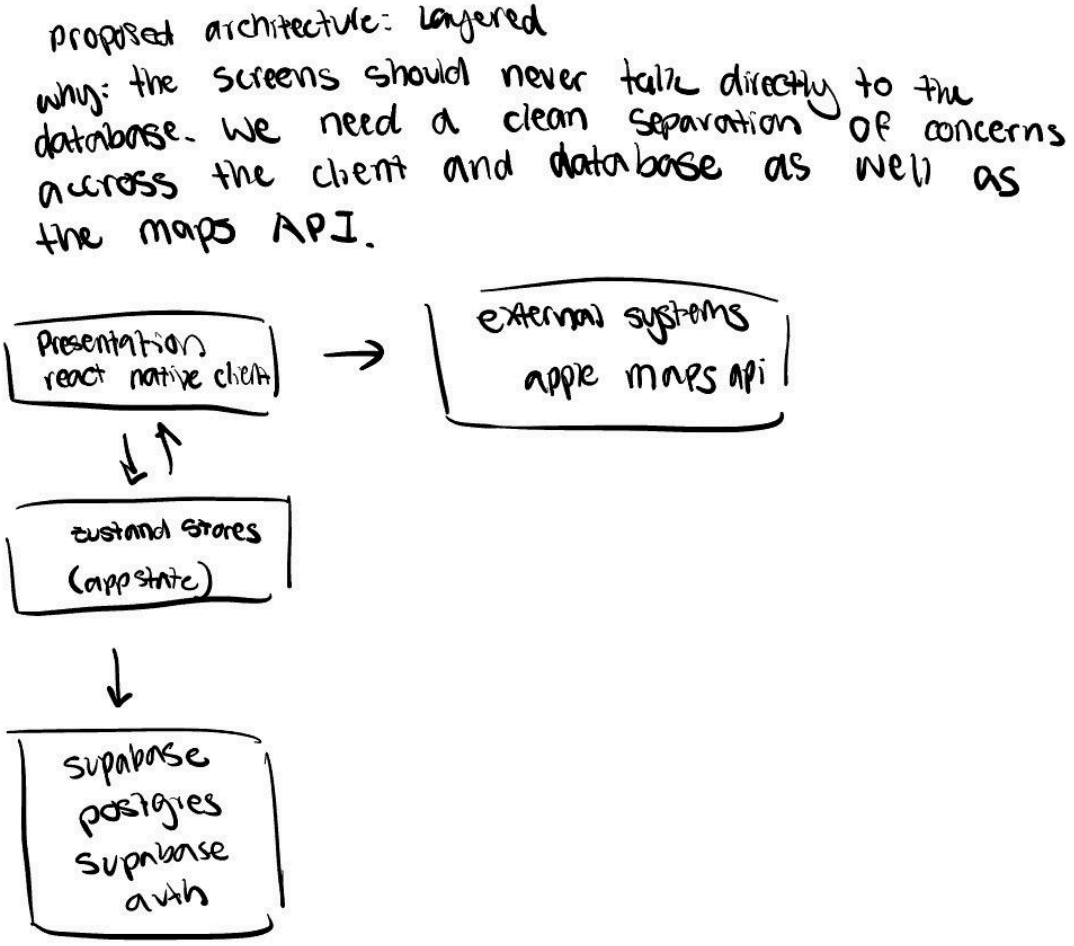
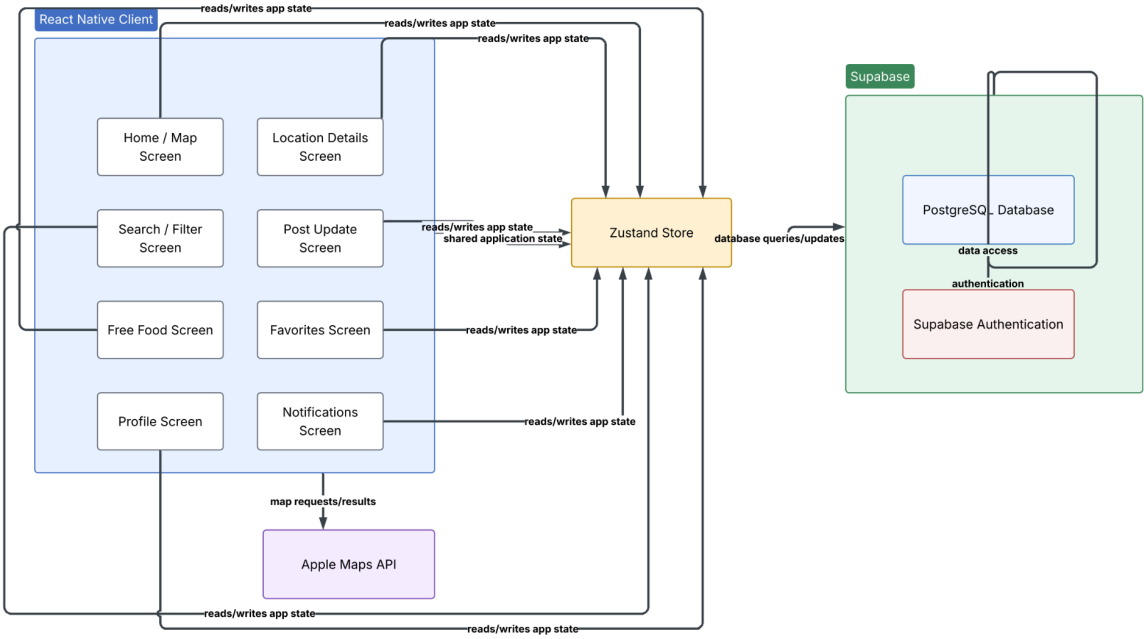
1. Proposed Architecture



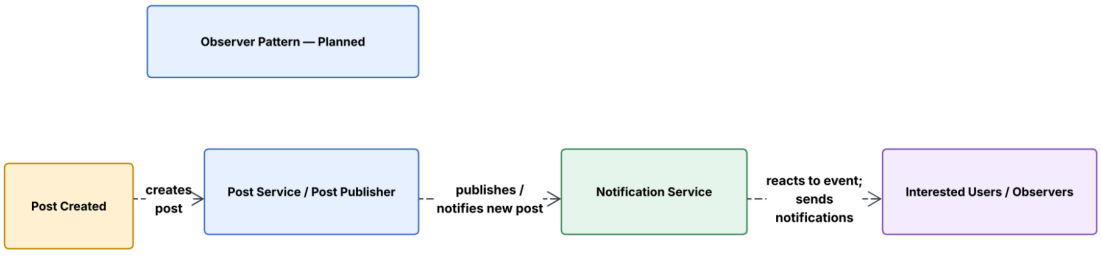
Musty Map uses a layered architecture. The React Native client handles the user interface, Zustand manages shared application state, and Supabase provides persistent PostgreSQL data storage and authentication. Apple Maps is used as an external system for map functionality. This architecture fits Musty Map because it separates the user interface, application state, persistent data, authentication, and external map functionality. The screens do not need to contain all of the database and application logic themselves, which makes the application easier to maintain and change as more features are added.

2. High-Level Design



The high-level design shows the major parts of Musty Map and how data moves between them. The React Native client contains the main screens users interact with, including the map, location details, search and filters, posts, favorites, profiles, and notifications. These screens use the Zustand store to access and update shared application state.
Zustand communicates with Supabase for persistent data and authentication. Supabase stores information such as users, locations, posts, food updates, favorites, and notifications. Apple Maps is an external service used by the client for map and location functionality.

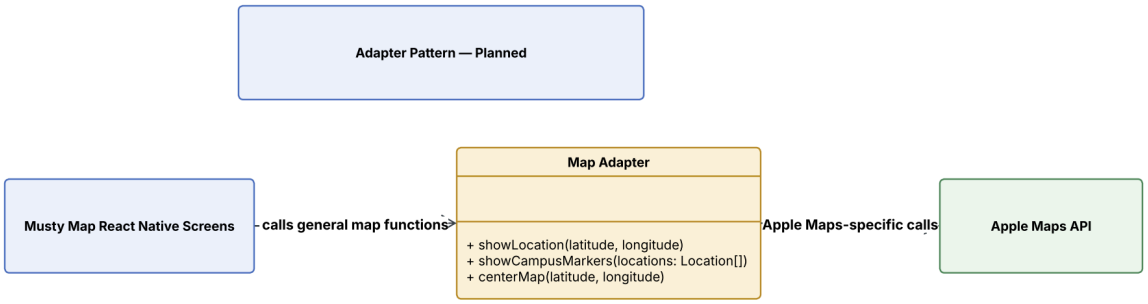
3. Design Patterns



Musty Map plans to use the Observer pattern for notifications. When a user creates a new post, other users who are interested in that activity may need to receive a notification.
The main components involved are the Post Service, Notification Service, and interested users. The Post Service acts as the publisher of the event. When a post is created, the Notification Service reacts and sends notifications to the appropriate users.

Using the Observer pattern keeps notification logic separate from posting logic. Without this pattern, notification behavior could end up being placed directly inside post screens or other large components, making the application harder to change and maintain.

a. Adapter pattern–Planned



Musty Map plans to use the Adapter pattern for Apple Maps. The Map Adapter gives the React Native screens general map functions while keeping Apple Maps-specific calls separate from the rest of the application.
The main components involved are the React Native screens, Map Adapter, and Apple Maps API. Using an Adapter keeps the app from being tightly connected to Apple Maps-specific code and makes the map functionality easier to change later.

4. Design Decisions and Tradeoffs

Supabase vs. Building Our Own Backend

We considered building authentication and database services ourselves or using Supabase. We chose Supabase because it already provides PostgreSQL storage and authentication, which saves development time. The tradeoff is that the app depends more on Supabase.

Zustand vs. Managing State Separately in Each Screen

We considered managing shared data inside each React Native screen or using Zustand. We chose Zustand because multiple screens need access to the same data, such as users, locations, posts, and favorites. The tradeoff is that it adds another layer to the app.